

Programação Orientada a Objetos 2025/2026

- Relatório de Projeto -

Jogo da Memória

Turma:	Número:	Nome:
PL8	2024151350	Diogo Brito
PL8	2024151531	Rafael Junqueira
Docente Orientador:		
Prof(a). Patrícia Macedo		

Data: 06/2026

Índice

Apresentação do Projeto.....	3
Modelação do Domínio	4
Diagrama de Classes UML	5
Arquitetura da Aplicação	6
Funcionalidades Implementadas	7
Aplicação dos Conceitos de Programação Orientada a Objetos	12
Tratamento de Exceções	13
Testes Unitários	14
Dificuldades Encontradas.....	15
Conclusão	16

Apresentação do Projeto

Este projeto foi desenvolvido no âmbito da Unidade Curricular de Programação Orientada a Objetos, do curso de Licenciatura em Engenharia Informática e consiste na implementação de um Jogo da Memória em Java, com interface gráfica desenvolvida com recurso do **JavaFX**.

O objetivo do jogo é encontrar todos os pares de cartas existentes num tabuleiro definido por níveis de dificuldade (**Fácil**, **Médio** e **Difícil**) antes de esgotar o número de tentativas disponíveis. Em cada jogada, o utilizador seleciona duas cartas; se estas formarem um par, são removidas visualmente do tabuleiro, caso contrário, voltam a ficar escondidas. O jogo inclui ainda cartas especiais, capazes de produzir efeitos que alteram temporariamente o estado da partida.

A interação com o utilizador é realizada através de uma interface gráfica intuitiva, que permite selecionar o nível de dificuldade, visualizar o tabuleiro, revelar cartas e acompanhar informações relevantes, como o número de tentativas restantes e os pares encontrados.

Modelação do Domínio

A entidade central é o **GameEngine**, responsável por controlar a lógica da aplicação, validar jogadas, contabilizar tentativas e verificar as condições de vitória.

O **Board** representa a grelha onde as cartas são distribuídas. É responsável pela criação, armazenamento e organização das cartas, permitindo o acesso às mesmas através das respetivas coordenadas.

A classe abstrata **Card** representa o conceito genérico de carta, definindo características comuns como o símbolo associado e o estado atual da carta.

A partir desta classe foram criadas duas especializações:

- **NormalCard**, que representa as cartas tradicionais do jogo;
- **SpecialCard**, que representa cartas com efeitos especiais capazes de alterar o comportamento normal da partida.

O sistema inclui ainda o *enum* **GameDifficulty**, responsável por representar os diferentes níveis de dificuldade disponíveis. Cada nível define um conjunto de configurações do jogo, incluindo o número de linhas e colunas do tabuleiro, a quantidade de cartas especiais e o número inicial de tentativas. Esta estrutura é utilizada durante a criação de uma nova partida para parametrizar o tabuleiro e o motor do jogo.



Arquitetura da Aplicação

Modelo de domínio

O modelo de domínio encontra-se no pacote *model* e contém as classes responsáveis pela lógica principal do jogo.

O modelo de domínio concentra toda a lógica de negócio do jogo, sendo constituído por classes como **Board**, **Card**, **NormalCard**, **SpecialCard** e **GameEngine** bem como pelo *enum* **GameDifficulty**. Estas classes são responsáveis pela representação do estado do jogo, gestão das cartas, validação das jogadas e controlo das regras definidas.

Controlo da aplicação

O controlo da aplicação encontra-se no pacote *controller* e é responsável por fazer a ligação entre a interface gráfica e o modelo de domínio.

- A classe **StartController** controla o ecrã inicial da aplicação. Esta classe permite ao utilizador escolher a dificuldade do jogo e iniciar uma nova partida. Com base na dificuldade selecionada, são criados o tabuleiro e o motor do jogo;
- A classe **GameController** controla o ecrã principal do jogo. Esta classe recebe as ações do utilizador, como a seleção de cartas, invoca os métodos do **GameEngine** e atualiza a interface gráfica através da **GameView**.

Interface gráfica JavaFX

A interface gráfica foi desenvolvida utilizando **JavaFX**, proporcionando uma interação visual simples e intuitiva. Através da interface, o utilizador pode iniciar um novo jogo, selecionar cartas, acompanhar o número de tentativas realizadas e visualizar o estado atual do tabuleiro.

- A classe **StartView** representa o ecrã inicial do jogo, onde o utilizador pode escolher a dificuldade e iniciar a partida;
- A classe **GameView** representa o ecrã principal do jogo, apresentando o tabuleiro, as cartas, o número de tentativas, o número de pares encontrados e as mensagens de estado.

Funcionalidades Implementadas

Durante o desenvolvimento do projeto foram implementadas diversas funcionalidades essenciais para o correto funcionamento do jogo, por exemplo:

- Distribuição aleatória das cartas no tabuleiro;

```
private void createCards() { // Usage: A 2024101350 Diego Brito v1
    List<Card> cardList = new ArrayList<>();

    int numberOfPairs = getTotalNormalPairs();

    for (int i = 0; i < numberOfPairs; i++) {
        String symbol = SYMBOLS[i];

        cardList.add(new NormalCard(symbol));
        cardList.add(new NormalCard(symbol));
    }

    for (int i = 0; i < specialCards; i++) {
        if (i % 2 == 0) {
            cardList.add(new SpecialCard( symbol, SpecialCard.EffectType.BONUS));
        } else {
            cardList.add(new SpecialCard( symbol, SpecialCard.EffectType.SHUFFLE));
        }
    }

    Collections.shuffle(cardList, new Random());

    int index = 0;

    for (int row = 0; row < rows; row++) {
        for (int col = 0; col < cols; col++) {
            cards[row][col] = cardList.get(index);
            index++;
        }
    }
}
```

Figura 2 – Método createCards()

- Revelação e ocultação de cartas;

```
public void play(int row, int col) throws InvalidMoveException { // Usage: A 2024101350 Diego Brito v1
    if (isGameOver()) {
        throw new InvalidMoveException("O jogo já terminou.");
    }

    Card selectedCard = board.getCard(row, col);

    if (selectedCard.isFixed()) {
        throw new InvalidMoveException("Esta carta já foi selecionada.");
    }

    if (selectedCard.isRevealed()) {
        throw new InvalidMoveException("Esta carta já foi revelada.");
    }

    if (selectedCard instanceof SpecialCard && firstSelected != null) {
        firstSelected.hide();
        firstSelected = null;
    }

    selectedCard.reveal( gameEngine, this);

    if (selectedCard instanceof SpecialCard) {
        selectedSpecialCard = (SpecialCard) selectedCard;
    } else if (firstSelected == null) {
        firstSelected = selectedCard;
    } else {
        secondSelected = selectedCard;
        attempts--;
    }
}
```

Figura 3 – Método play()

- Implementação de cartas especiais com efeitos adicionais;

```
/**
 * Aplica o efeito da carta especial uma única vez.
 *
 * @param gameEngine motor do jogo
 */
public void applyEffect(GameEngine gameEngine) { 1 usage 2024151350 Diogo Brito +1
    if (!effectApplied) {
        executeEffect(gameEngine);
        effectApplied = true;
    }
}

/**
 * Executa o efeito da carta especial.
 *
 * @param gameEngine motor do jogo
 */
private void executeEffect(GameEngine gameEngine) { 1 usage 2024151350 Diogo Brito +1
    if (effectType == EffectType.BONUS) {
        gameEngine.addAttempts( value: 3);
    } else if (effectType == EffectType.SHUFFLE) {
        gameEngine.shuffleUnfixedCards();
    }
}
```

Figura 4 – Métodos *applyEffect()* e *executeEffect()*

Programação Orientada a Objetos – Jogo da Memória

- Interface gráfica interativa desenvolvida em JavaFX;



Figura 5 – Menu Principal do Jogo

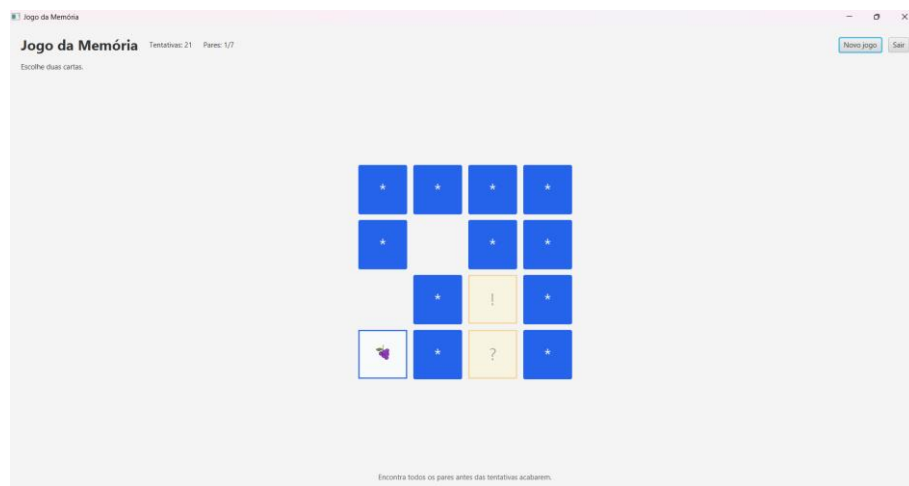


Figura 6 – Exemplo de jogabilidade e gráficos do jogo

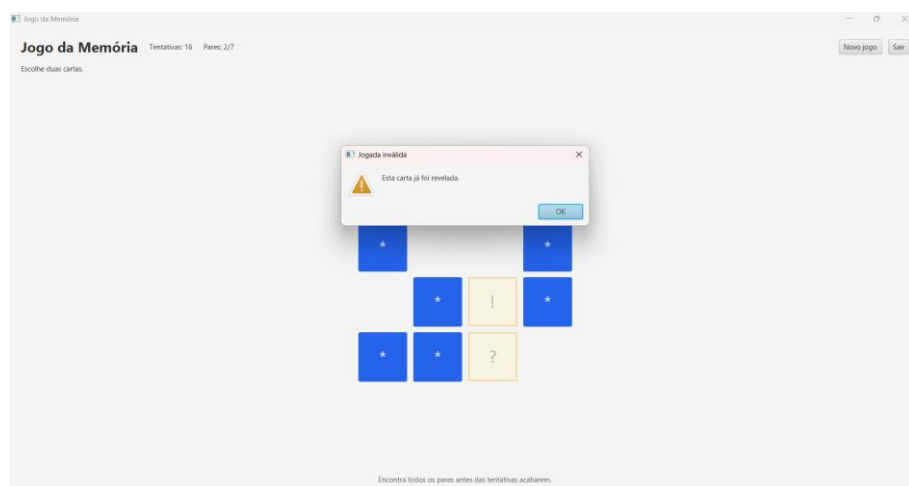


Figura 7 – Mensagens de informação ao utilizador

Programação Orientada a Objetos – Jogo da Memória

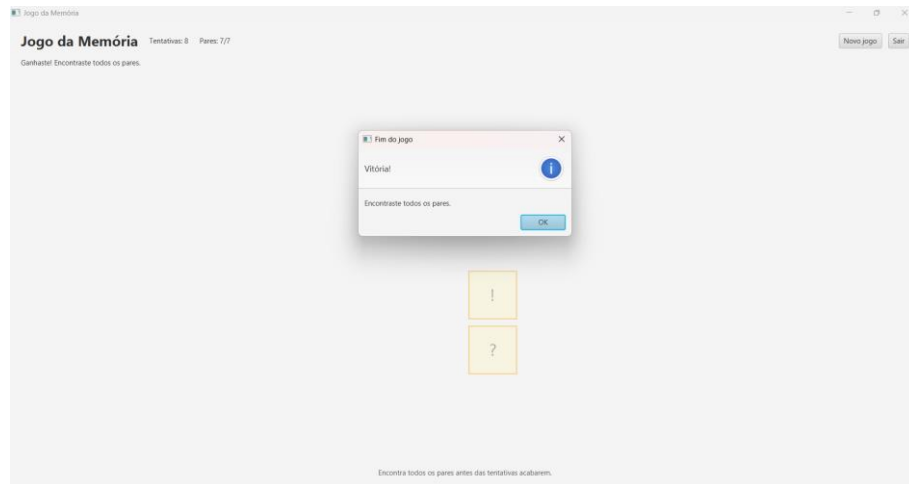


Figura 8 – Mensagem e interface de vitória

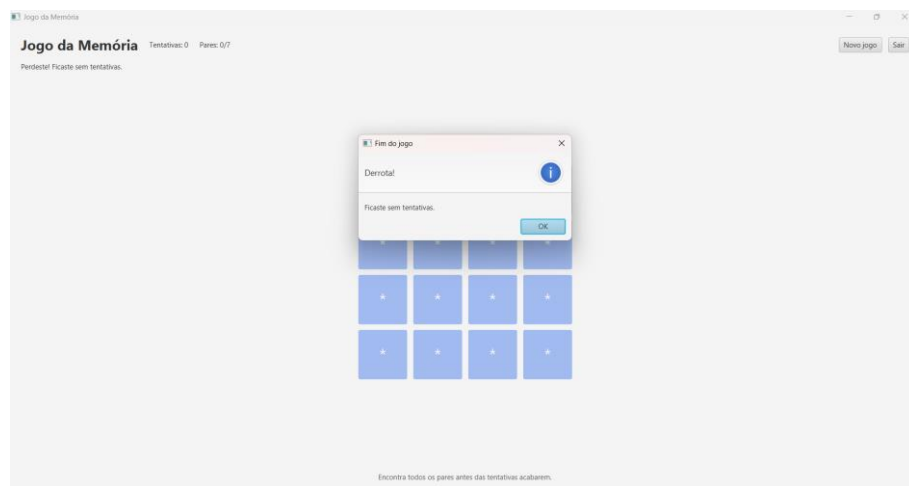


Figura 9 – Mensagem e interface de derrota

- Tratamento de situações inválidas através de exceções específicas.

```
private Board(int rows, int cols, int specialCards, int initialAttempts) throws InvalidBoardException {
    if (rows <= 0 || cols <= 0) {
        throw new InvalidBoardException("Dimensão do tabuleiro tem que ser positiva.");
    }
    if ((rows * cols) % 2 != 0) {
        throw new InvalidBoardException("Tabuleiro tem que ter um número par de posições.");
    }
    if (specialCards <= 0 || specialCards % 2 != 0) {
        throw new InvalidBoardException("Número de cartas especiais inválido.");
    }
    if (specialCards >= rows * cols) {
        throw new InvalidBoardException("O tabuleiro precisa de cartas normais.");
    }
    if (getTotalNormalPairs(rows, cols, specialCards) > SYMBOLS.length) {
        throw new InvalidBoardException("Não existem imagens suficientes para este tabuleiro.");
    }

    this.rows = rows;
    this.cols = cols;
    this.specialCards = specialCards;
    this.initialAttempts = initialAttempts;
    this.cards = new Card[rows][cols];

    createCards();
}
```

```
public void play(int row, int col) throws InvalidMoveException { 1 usage  2024151350 Diogo Brito +1
    if (isGameOver()) {
        throw new InvalidMoveException("O jogo já terminou.");
    }

    Card selectedCard = board.getCard(row, col);

    if (selectedCard.isFixed()) {
        throw new InvalidMoveException("Esta carta já foi selecionada.");
    }

    if (selectedCard.isRevealed()) {
        throw new InvalidMoveException("Esta carta já foi revelada.");
    }

    if (selectedCard instanceof SpecialCard && firstSelected != null) {
        firstSelected.hide();
        firstSelected = null;
    }
}
```

Figuras 10 e 11 – Tratamento de comportamentos inválidos

Aplicação dos Conceitos de Programação Orientada a Objetos

Encapsulamento

O encapsulamento foi aplicado através da definição dos atributos das classes como privados, garantindo que o acesso aos dados internos apenas é efetuado através de métodos públicos apropriados, aumentando a segurança da aplicação e reduzindo a possibilidade de estados inconsistentes.

Herança

A herança foi utilizada para especializar diferentes tipos de cartas. A classe abstrata **Card** funciona como classe base, sendo posteriormente estendida pelas classes **NormalCard** e **SpecialCard**. Desta forma foi possível reutilizar código comum e simplificar a implementação de comportamentos específicos.

Polimorfismo

O polimorfismo permitiu tratar os dois tipos de cartas (**NormalCard** e **SpecialCard**) através de referências do tipo **Card**, reduzindo o acoplamento e aumentando a coesão entre as diferentes componentes do sistema.

Tratamento de Exceções

InvalidBoardException

A exceção **InvalidBoardException** é utilizada na classe **Board** para impedir a criação de tabuleiros inválidos. Esta exceção é lançada quando as dimensões do tabuleiro não são positivas, quando o número total de posições é ímpar, quando o número de cartas especiais é inválido, quando o tabuleiro não possui espaço suficiente para cartas normais ou quando não existem imagens suficientes para preencher o tabuleiro.

InvalidMoveException

A exceção **InvalidMoveException** é utilizada para validar jogadas inválidas. Esta exceção é lançada, por exemplo, quando o utilizador tenta aceder a uma posição fora dos limites do tabuleiro, seleccionar uma carta já fixa, seleccionar uma carta já revelada ou jogar depois de o jogo já ter terminado.

Para além destas exceções, foi também utilizada a exceção **IllegalArgumentException** em situações de argumentos inválidos, por exemplo, quando se tenta criar uma carta com símbolo nulo ou vazio, criar uma carta especial sem tipo de efeito, criar um **GameEngine** com tabuleiro nulo ou adicionar um número de tentativas menor ou igual a zero.

Testes Unitários

Na classe **BoardTest** foram testados cenários relacionados com a criação do tabuleiro. Foi verificada a criação de um tabuleiro válido com 4 linhas e 4 colunas, bem como situações inválidas, como dimensões negativas e tabuleiros com número ímpar de posições no qual seria lançada a exceção **IllegalArgumentException**.

Na classe **CardTest** foram testados comportamentos comuns às cartas. Foi verificado que uma carta normal começa escondida e não fixa com o método **isRevealed()**, que o método **fix()** altera corretamente o estado da carta para fixa e revelada, e que a criação de uma carta com símbolo inválido lança **IllegalArgumentException**.

Na classe **NormalCardTest** foi testado o método **reveal()**, confirmando que uma carta normal passa para o estado revelado quando este método é chamado.

Na classe **SpecialCardTest** foram testadas validações associadas às cartas especiais, nomeadamente a impossibilidade de criar uma carta especial com tipo de efeito nulo. Para este cenário foi utilizado **assertThrows()** para verificar o lançamento de **IllegalArgumentException**.

Na classe **GameEngineTest** foram testados aspetos relacionados com o motor do jogo. Foi verificada a criação correta do **GameEngine**, o funcionamento do método **addAttempts()**, que aumenta o número de tentativas disponíveis, e a validação com **IllegalArgumentException** que impede a criação de um motor de jogo com tabuleiro nulo.

Dificuldades Encontradas

Ao longo do desenvolvimento do projeto foram sentidas algumas dificuldades, principalmente na implementação da interface gráfica utilizando **JavaFX**. Uma das maiores dificuldades esteve relacionada com a organização e posicionamento dos componentes gráficos, bem como na atualização dinâmica da interface em função das ações realizadas pelo utilizador durante o jogo.

Foram também encontrados desafios na implementação da lógica do jogo, nomeadamente na sincronização entre a lógica da aplicação e a interface gráfica.

Outra dificuldade relevante esteve relacionada com o tratamento de exceções em tempo de execução (*runtime*), particularmente na identificação e prevenção de situações que poderiam originar estados inválidos do jogo ou comportamentos inesperados da aplicação.

Para ultrapassar estas dificuldades, foi realizada pesquisa em diversas fontes de informação, websites especializados como o **Stack Overflow** e o **W3Schools**, vídeos tutoriais disponibilizados no **YouTube** e os materiais de apoio fornecidos na unidade curricular, nomeadamente os slides e exemplos práticos abordados nas aulas. Adicionalmente, recorreu-se ao auxílio de ferramentas de **Inteligência Artificial** para a clarificação de conceitos e otimização de soluções.

Stack Overflow

<https://stackoverflow.com/questions>

W3Schools

<https://www.w3schools.com/>

Conclusão

Ao longo do trabalho foram implementadas diversas funcionalidades, incluindo a criação dinâmica do tabuleiro, a gestão das cartas normais e especiais, a deteção de pares, o controlo do número de tentativas disponíveis e a interação com o utilizador através de uma interface gráfica intuitiva.

Durante o desenvolvimento do projeto foram aplicados conceitos importantes abordados na unidade curricular, nomeadamente encapsulamento, herança, polimorfismo, tratamento de exceções e testes unitários.

Este projeto permitiu aprofundar e relembrar a utilização da *framework* **JavaFX** de uma forma mais abrangente. A construção da interface gráfica, a gestão de eventos, a atualização dinâmica dos componentes visuais e a interação entre a interface e a lógica da aplicação permitiram adquirir uma melhor compreensão.